

Supplementary Materials for “SpliTEE: Fast and Private LLM Inference by Coupling GPU-Assisted Trusted Execution Environments with Differential Privacy”

Shashie Dilhara Batan Arachchige
Hassan Jameel Asghar

Robin Carpentier
Dali Kaafar

Contents

A1	Hyperlinks to Resources	2
A1.1	Language Models	2
A1.2	Datasets	2
A1.3	Others	2
A2	Implementation Details	2
A2.1	Host and TDX Configuration	2
A2.2	Shared Memory Communication Between TDX Guest and GPU Worker	2
A2.3	Noise Bank Design and Details	3
A2.4	Inference Model Architecture Details	3
A3	Restoration System Prompt	4
A4	Reconstruction Attacker Implementation Details	4
A4.1	Composite Attacker details	5

This document accompanies the manuscript “SpliTEE: Fast and Private LLM Inference by Coupling GPU-Assisted Trusted Execution Environments with Differential Privacy” (under review).

A1 Hyperlinks to Resources

A1.1 Language Models

Gemma models: <https://huggingface.co/collections/google/googles-gemma-models-family>

Mistral models: <https://huggingface.co/mistralai>

Qwen3-4B-Instruct: <https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507>

GPT 5.4 API <https://developers.openai.com/api/docs/models/gpt-5.4>

all-mpnet-base-v2 <https://huggingface.co/sentence-transformers/all-mpnet-base-v2>

A1.2 Datasets

PILE <https://huggingface.co/datasets/EleutherAI/pile>

Databricks Dolly <https://huggingface.co/datasets/databricks/databricks-dolly-15k>

A1.3 Others

Slalom <https://github.com/ftramer/slalom>

A2 Implementation Details

A2.1 Host and TDX Configuration

The system is equipped with an Intel Xeon Silver 4514Y processor with 16 physical cores and approximately 117 GB of system memory. The TDX-protected virtual machine is configured with 32 virtual CPUs and approximately 93 GB of memory, while the remaining system memory is available to the host. The host is equipped with an NVIDIA RTX PRO 4500 Blackwell GPU with approximately 32 GB of device memory. Both environments run Ubuntu 24.04.

A2.2 Shared Memory Communication Between TDX Guest and GPU Worker

Intel TDX provides private and shared memory for the protected virtual machine. Private memory is encrypted and integrity protected and can only be accessed from within the TDX environment, while shared memory is used to communicate with entities outside the TDX guest and does not receive the same protection [1]. Our TDX guest runs under QEMU (Quick Emulator), which provides Inter-VM Shared Memory (IVSHMEM)¹ between the host and guest. The masked input to each outsourced linear operation and the corresponding GPU output are exchanged through this region, as shown in Figure 3 (see Section A2.2 for additional details on the shared memory implementation).

¹See <https://www.qemu.org/docs/master/specs/ivshmem-spec.html>

The shared-memory region is implemented so that both the host and the TDX guest can access the same memory area. The host maps the shared-memory object into its address space, while the TDX guest maps the corresponding IVSHMEM device memory region into its own address space. Both sides can therefore access the same memory area, allowing large data transfers without serializing or transmitting them through the socket connection. Note that the shared-memory region contains only the masked input, and the corresponding output on the masked input from the GPU. The unmasked input, the mask, and the corresponding correction, are not written to the shared-memory region and remain inside the TDX guest. All masks and corrections are precomputed inside the protected virtual machine of the TDX guest, and written to its local disk. During inference, the required entries are loaded from this storage into the TDX guest’s protected memory, i.e., private memory.

Because the shared memory region is used only for exchanging the masked input and output, a separate mechanism is required to coordinate each outsourced computation. For this purpose, we use a TCP socket as a control channel between the TDX guest and the host. Control messages specify the metadata needed to locate and interpret the masked tensor in shared memory. The projection name identifies the requested operation, while the transformer-layer index identifies the correct weight matrix, since each layer has its own distinct set of weights. The shared-memory offset specifies where the masked tensor begins. Because the shared memory region holds only raw bytes with no inherent structure, its shape and datatype are also included, so the GPU worker can correctly interpret the stored values as a tensor. The byte length is included to verify that the expected amount of data is being accessed. Therefore, the masked input and corresponding output are transferred through IVSHMEM, while the TCP socket is used only to coordinate the outsourced computation.

A2.3 Noise Bank Design and Details

We maintain separate correction banks for (Q, K, V) projections, the O projection, the Gate/Up projections, and the language head projection. The (Q, K, V) projections receive the same input (see Figure 2). A single mask can therefore be applied to their shared input, with three corresponding corrections stored for the three weight matrices. Similarly, the Gate and Up projections share the same input and therefore need a single mask, with two corrections corresponding to the two matrices. The O projection and language model head use one mask and one correction each. The noise bank is *indexed* by the input shape. As mentioned in Section 3.2, the matrices have different dimensions, resulting in outputs of different dimensions (also called tensor dimensions). Thus, during inference, the TEE retrieves an entry matching the current tensor dimensions, which serves as its index. Each noise entry, the mask and its corresponding correction, is consumed only once, preventing reuse of the same mask across different calls to these components. All masks and corrections are precomputed inside the TDX protected virtual machine and written to its local disk. Before inference, the entries required for the current batch are loaded from disk into protected memory. During inference, these loaded entries are used one at a time and discarded after use.

A2.4 Inference Model Architecture Details

Llama-3B contains 28 transformer layers with a hidden dimension of 3,072 and an intermediate MLP dimension of 8,192. Its attention module uses 24 query heads and 8 key-value heads. The Q and O projections operate over 3,072-dimensional representations, while the K and V projections produce 1,024-dimensional outputs. The Gate and Up projections expand the hidden representation to 8,192 dimensions. Its LM head maps the final 3,072-dimensional representation to a vocabulary of 128,256

tokens.

Qwen-4B contains 36 transformer layers with a hidden dimension of 2,560 and an intermediate MLP dimension of 9,728. It uses 32 query heads and 8 key-value heads. Its Q projection produces a 4,096-dimensional representation, while K and V each produce 1,024 dimensions, and the O projection maps the 4,096-dimensional attention output back to the 2,560-dimensional hidden representation. The Gate and Up projections expand the representation to 9,728 dimensions, and the LM head maps the final hidden representation to a vocabulary of 151,936 tokens.

A3 Restoration System Prompt

System Prompt

You are a precise text restoration assistant. The input was reconstructed from neural activations and may contain repetition, corrupted words, missing words, grammar errors, spacing errors, and incomplete fragments. Restore the entire text into the closest readable and coherent version while preserving all recoverable content and specific wording. Merge repeated fragments into one complete version and repair broken sentence structure, grammar, word order, and spacing whenever this can be done from the available context. If a short function word or grammatical connector is clearly missing, you may restore it when needed to make the sentence grammatically correct, but do not introduce new facts, entities, claims, or details that are not supported by the input. Do not summarize, shorten, or omit names, numbers, places, technical terms, questions, or contextual details. The final text must be readable, coherent, and grammatically correct while remaining as faithful as possible to the information present in the reconstructed text. Do not answer the prompt. Return only the restored text.

A4 Reconstruction Attacker Implementation Details

This appendix provides the implementation details of the reconstruction attacker introduced in Section 7.4.1.

The reconstruction attacker operates on intermediate representations obtained from the selected outsourced operator in Layer 0. The dimensionality of the intermediate representation depends on the selected operator and capture point. For example, the input to the (Q, K, V) projections in Layer 0 has a dimension of 3072. These hidden states cannot be passed directly to the attacker model in the same way as normal token embeddings. Although they may have the same dimension, they represent internal features produced by the victim model rather than the attacker model’s input-embedding space. We therefore use a small trainable projection network, which acts as a learned mapping between these two representations.

The projection network first expands each intermediate vector into a higher-dimensional space so that it has more capacity to transform and reorganize the information. In our implementation, this intermediate space is four times the input dimension. A SiLU activation is then applied to introduce a nonlinear transformation. The representation is subsequently reduced back to the embedding dimension expected by the attacker model, followed by a second SiLU activation. Applying SiLU after both the expansion and reduction layers allows the projection network to capture more complex nonlinear relationships between the intermediate representation and the attacker model’s embedding space. Finally, RMSNorm is applied to keep the projected representation on a stable scale before it is passed to the attacker model.

The output of the projection network is then used as a *soft prefix* for the attacker model. Here, a soft prefix is a sequence of continuous embedding vectors produced by the projection network. It provides the initial context for generation, similar to a normal token prefix. Unlike a normal text prefix, it is not created from input token IDs. Instead it provides the attacker with the starting context needed to reconstruct the original prompt.

During training, the attacker is given the projected representation along with the original prompt as the target output. We use teacher forcing, where the correct preceding prompt tokens are provided to the model while it learns to predict the next token, rather than relying on its own previous predictions [2]. This helps the attacker learn how to reconstruct the prompt token by token from the information contained in the captured representation. The target sequence also includes an end-of-sequence (EOS) token so that the model learns when to stop generating.

To keep the reconstruction attacker lightweight, most of the pretrained attacker model is kept frozen during training. Instead of updating all model parameters, we use Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning method that introduces a small number of trainable parameters into selected layers while leaving the original model weights unchanged [3]. This allows the language model to adapt to the prompt-reconstruction task without requiring full model fine-tuning. At the same time, the projection network is trained to transform the captured intermediate representation into a soft prefix compatible with the attacker model’s embedding space. The projection network and LoRA adapters are trained jointly, allowing the attacker to learn how to interpret the captured representation and reconstruct the original prompt while updating only a small fraction of the overall model parameters. The reconstruction model is trained for five epochs. The maximum target length used during teacher forcing is increased gradually, allowing the model to first learn shorter portions at the beginning of the prompt before being trained on the complete target sequence. After each epoch, the model is evaluated on a held-out validation set, and a checkpoint is saved. The checkpoint with the lowest validation loss is selected as the attacker model used in the final evaluation.

A4.1 Composite Attacker details

The three inputs to the (Q, K, V) projections, O projection and Gate/Up projections are aligned by token position and concatenated along the model’s dimension d . Since each input has tensor shape $[n, d]$, where n is the prompt length and d is the hidden dimension, the resulting observation has shape $[n, 3d]$. For the evaluated Llama-3B model, this corresponds to combining three $[n, 3072]$ tensors into a single $[n, 9216]$ tensor. The reconstruction model then separates the combined observation into its (Q, K, V) , the O projection, and Gate/Up components. Each component is processed by an independent projection branch, after which the projected representations (see Figure 5) are fused into a single soft prefix. This soft prefix is used to generate one reconstruction of the original prompt.

A couple of remarks are in order. We do not include the LM head in the composite attack because it does not improve reconstruction over the other three components. As the LM head appears at the end of the inference pipeline, it is also less directly related to the original prompt. We further restrict the attack to Layer 0, where the inputs are closest to the original prompt and have undergone the least transformation. Including deeper layers adds training complexity without substantially improving the attack.

References

- [1] Pau-Chen Cheng, Wojciech Ozga, Enriquillo Valdez, Salman Ahmed, Zhongshu Gu, Hani Jamjoom, Hubertus Franke, and James Bottomley. Intel tdx demystified: A top-down approach.

ACM Comput. Surv., 56(9), April 2024.

- [2] Ronald J. Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1(2):270–280, 1989.
- [3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.